Project Guidelines

The project is intended to verify that students can apply a modern computer-vision method, evaluate it correctly, interpret its behavior, and explain the results. The emphasis is on understanding and experimental reasoning.

Project principles:

1. Small, controlled scope: Use a public dataset or a small custom dataset. Large-scale model training is not required.
2. Pretrained models are encouraged: The scientific contribution is the experiment, comparison, evaluation, or failure analysis.
3. Common evidence: Every team must provide a baseline, a second condition/model, quantitative metrics, and a failure analysis.
4. Reproducible result: The submitted code/notebook must reproduce the reported result.
5. Individual understanding: Every student completes a short supervised project-understanding check, independent of the team grade.

Team Formation and Topic Assignment

- Teams consist of 4 students. A team of 3 is allowed only with instructor approval.
- Each team select a project from the provided list
- A project cannot be chosen by more than one team

The teams should pick project from the sheet and write their names next to it by 16.09.2026
After this date the teaching team will distribute the unallocated students randomly in teams and assign projects for them. And this will be considered mandatory and cannot be revised.

List of Project:

https://docs.google.com/spreadsheets/d/1G8_iV4xF9VyqWe4vlHy7VSjP9mxTog_O8kPNJ81sl-I/edit?usp=sharing

Required Deliverables

A. Git repository or notebook with clear run instructions, requirements/environment information, and no missing files.

B. Two-page technical summary. Include problem, dataset, method, experiment, key quantitative result, one compact figure/table, failure analysis, and each student contribution.

C. Five-minute demo: Demonstrate the task, the baseline, the main comparison, and one failure case. Maximum 3 presentation slides in addition to the live demo.

D. Q&A: The members, individually, will be asked to explain a model choice, metric, result, threshold, or failure case.

Common Experimental Requirements

- Define a clear research question or engineering question. Example: “Does a ViT remain more robust than a CNN under image corruption on this dataset?”
- Establish a baseline before introducing the proposed change.
- Change only one major factor at a time when possible: architecture, prompt, augmentation, threshold, loss, feature representation, or tracking method.
- Report at least two appropriate metrics. Examples: accuracy/F1, mAP, IoU/Dice, precision/recall, retrieval Recall@K, IDF1/HOTA, latency, or memory usage.
- Include one concise ablation or sensitivity experiment.
- Inspect at least 3 failure cases and explain likely causes rather than only displaying them.
- State the dataset split and prevent train/test leakage.
- Report runtime environment and the approximate inference/training time.

Project Grading

The project contributes 25% of the final course grade. 20% are shared by the team; 5% is individual.

Team component	Points	Evidence
Problem formulation and experimental question	2.0	Clear objective, appropriate scope, justified baseline
Correct implementation and reproducibility	4.0	Code runs and produces the reported output
Experimental design / comparison	2.0	Controlled comparison, meaningful variable
Evaluation and metrics	2.0	Correct metrics, readable table/figure, quantitative interpretation
Failure analysis and technical reasoning	4.0	Representative failures, plausible explanation, limitations
Demo quality and technical clarity	6.0	Five-minute demo is concise and answers the research question

5 points individual Q/A for each student

Two-Page Technical Summary Template

1. **Problem and question:** One or two sentences. State exactly what is being compared or evaluated.
2. **Dataset:** Name, size/subset, train/validation/test split, and any preprocessing.
3. **Method:** Baseline + second condition/model. Mention pretrained weights and important settings.
4. **Results:** One compact table or figure with the main metrics.
5. **Interpretation:** Two or three sentences explaining what the result means.
6. **Failure analysis:** Show or describe representative failures and likely causes.
7. **Reproducibility:** Repository link, environment, approximate runtime.

8. Members' Contributions: Clearly state each member's contribution to the project.

Demo Format (10 min)

- problem and dataset
- baseline and comparison
- quantitative result and one visual example
- failure case / limitation
- conclusion
- questions from teachers

Submission Checklist

- ☐ README with exact run instructions
- ☐ Notebook or source code
- ☐ requirements.txt / environment description
- ☐ Dataset download instructions
- ☐ Saved final metrics and figures
- ☐ Two-page technical summary
- ☐ Team member names and contributions